

Validation

Week 14

D4 Robotics Engineering Study Program
Department of Electrical Engineering
Politeknik Negeri Batam

Machine Learning

Bias and Variance

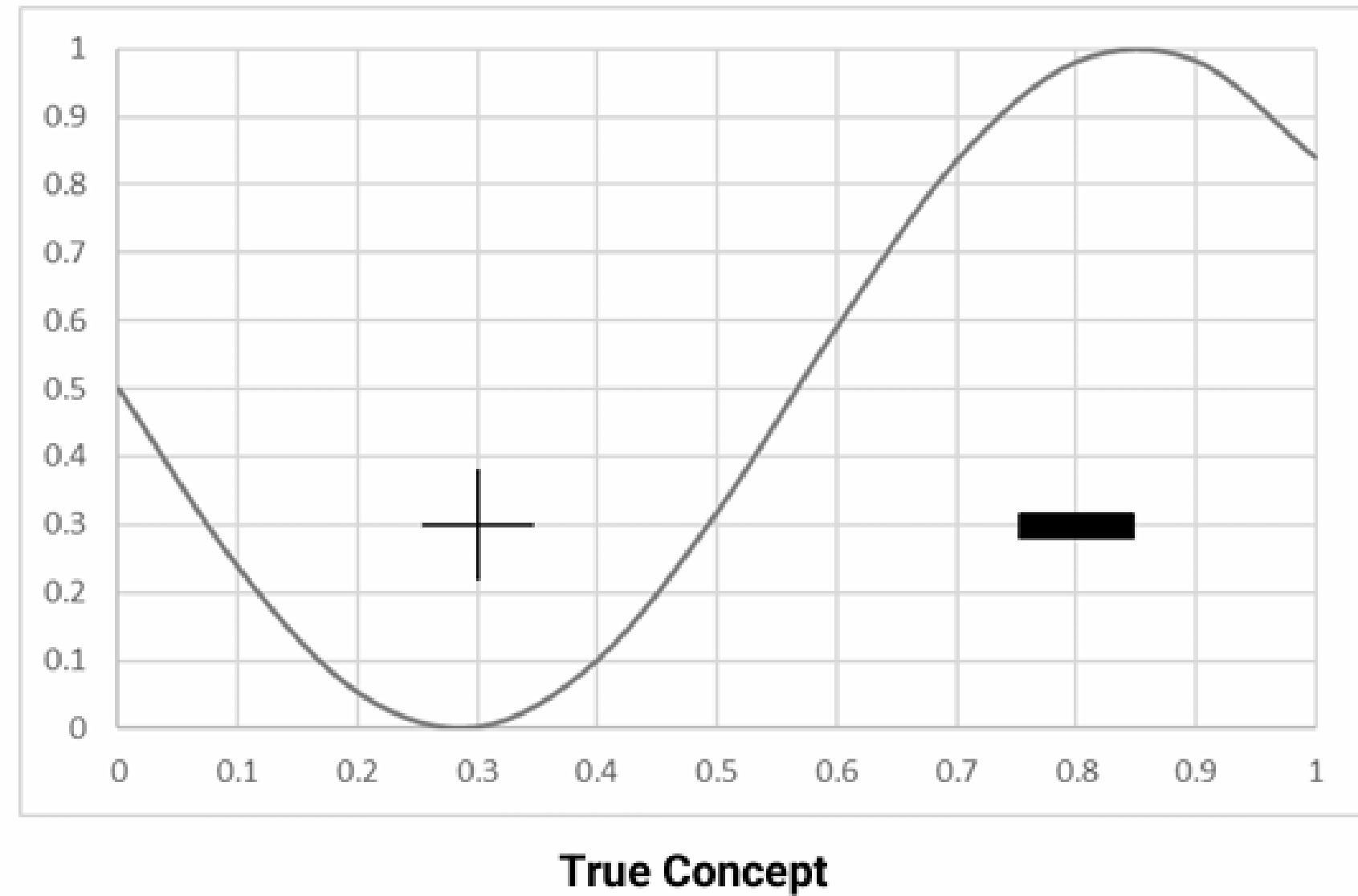


Konsep Dasar

- **Bias:** Error yang disebabkan karena model tidak dapat merepresentasikan konsep yang sebenarnya.
- **Variance:** Error yang disebabkan karena algoritma pembelajaran bereaksi berlebihan terhadap perubahan kecil (noise) dalam data training

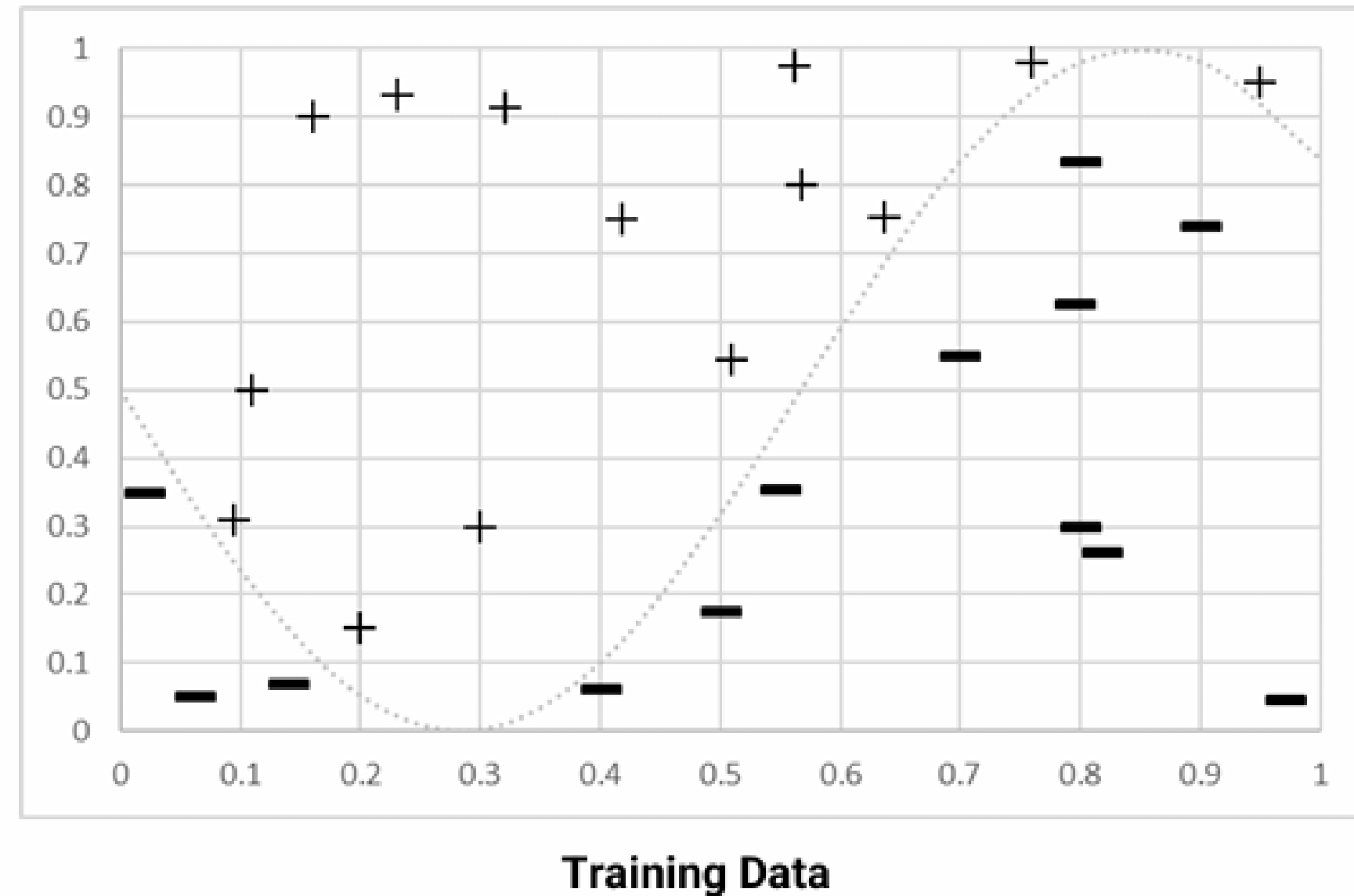
Bias and Variance

Goal: Menghasilkan model yang sesuai dengan konsep seperti gambar ini



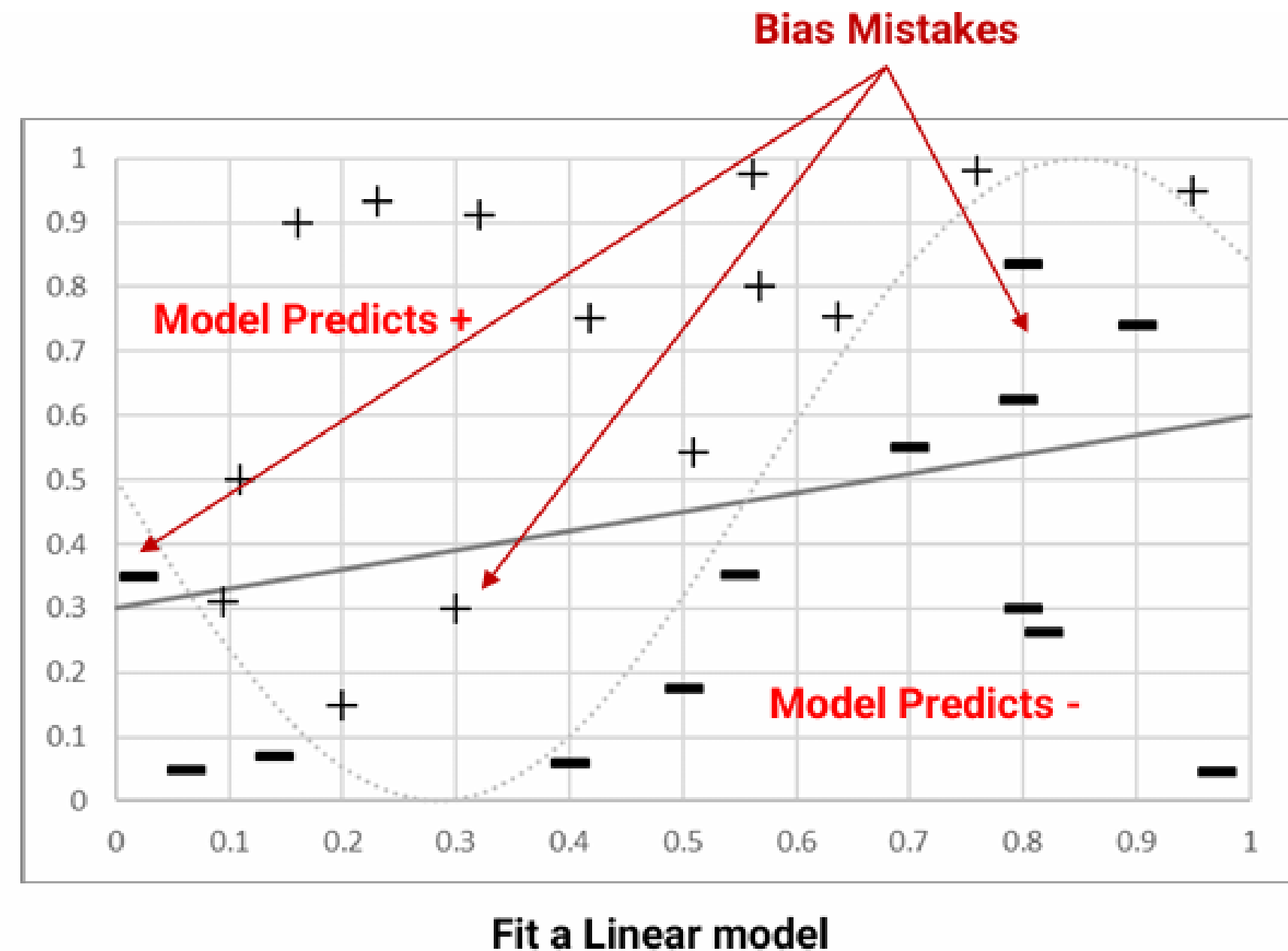
Visualizing Bias

- Goal: Menghasilkan model yang sesuai dengan konsep seperti gambar ini
- Data pelatihan untuk konsep diatas



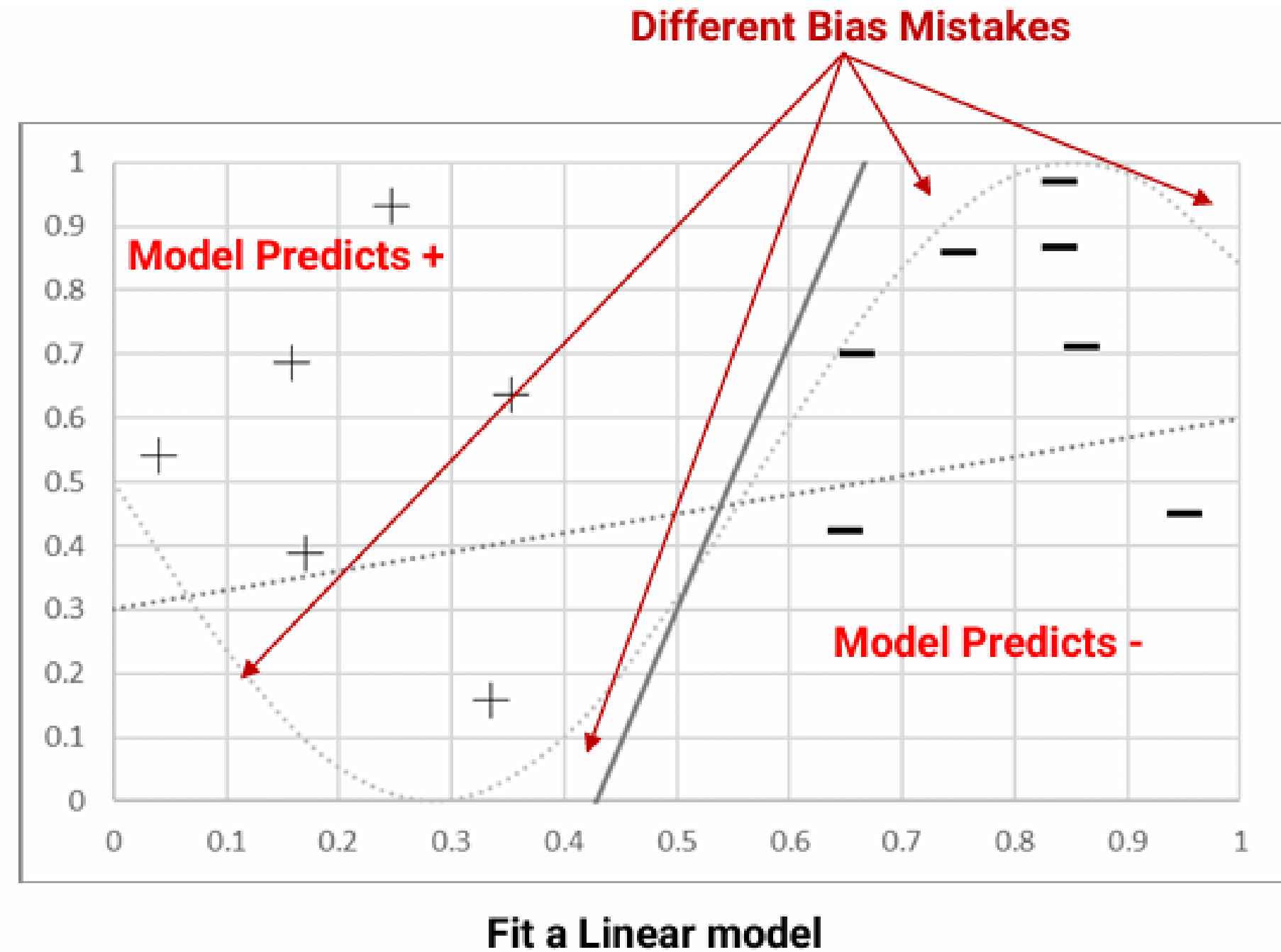
Visualizing Bias

- Goal: Menghasilkan model yang sesuai dengan konsep seperti gambar ini
- Data pelatihan untuk konsep diatas
- Bias: Tidak bisa mempresentasikannya



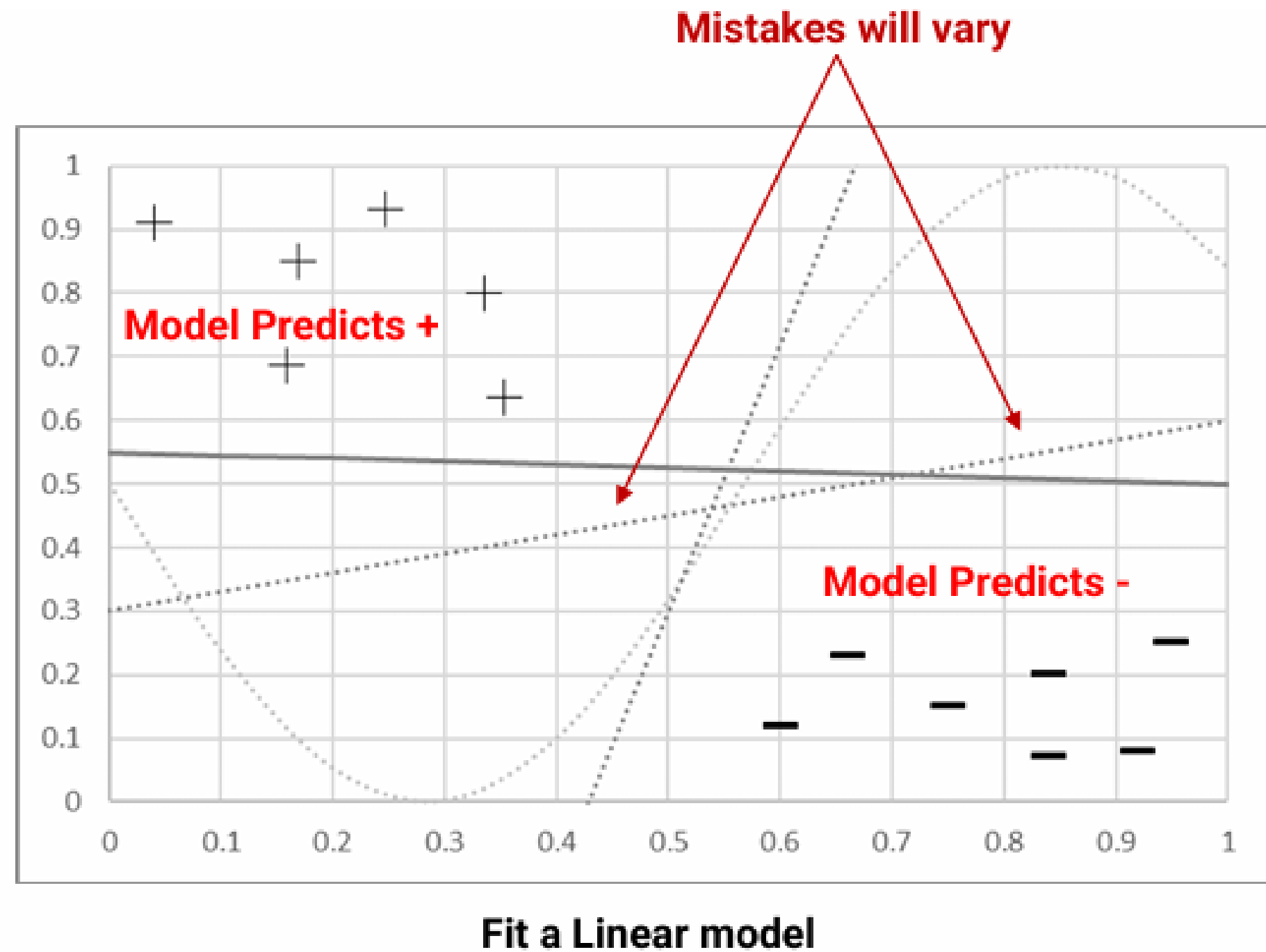
Visualizing Variance

- Goal: Menghasilkan model yang sesuai dengan konsep seperti gambar ini
- Data baru, model baru



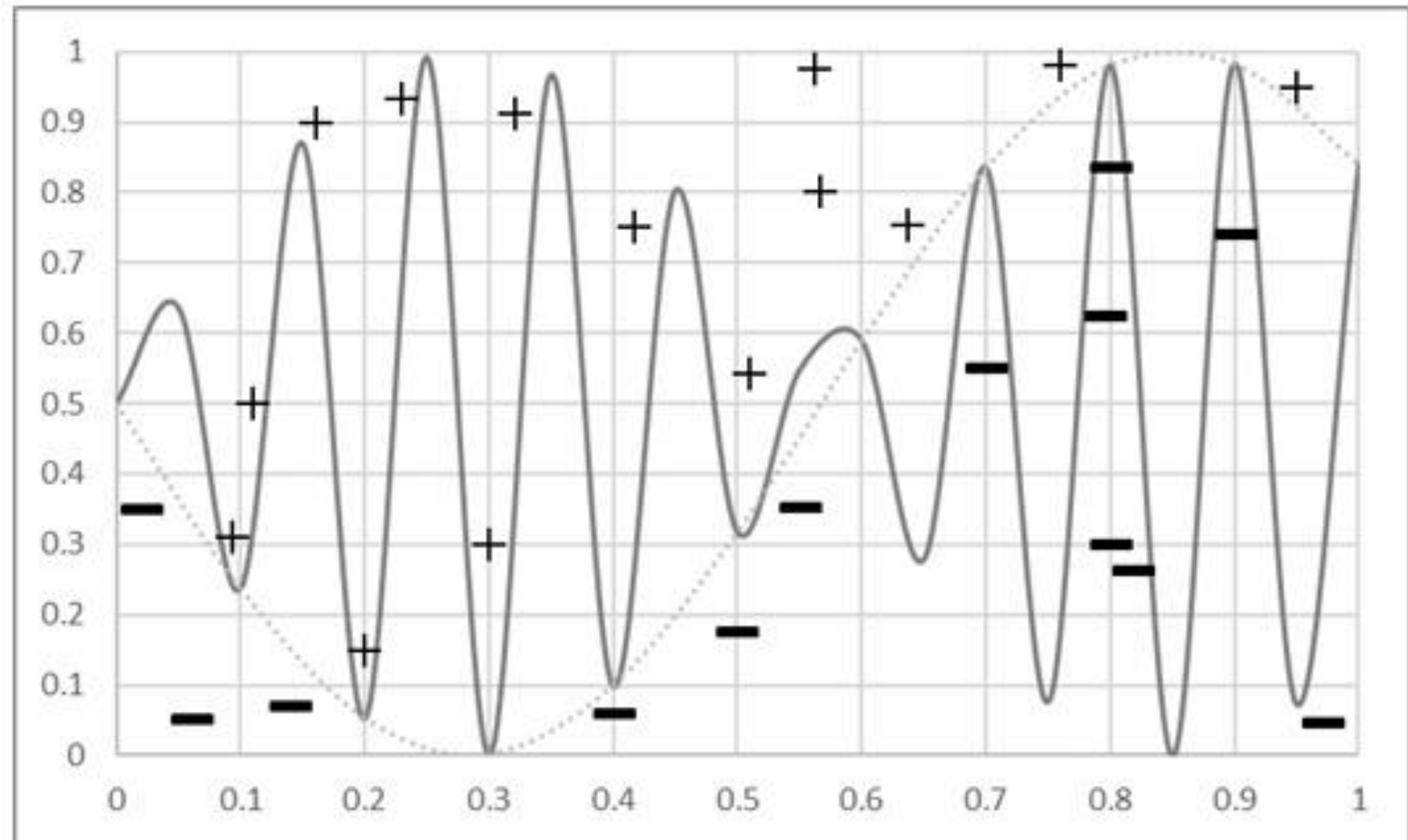
Visualizing Variance

- Goal: Menghasilkan model yang sesuai dengan konsep seperti gambar ini
- Data baru, model baru
- Data baru, model baru
- Varians: Sensitif terhadap perubahan dan noise



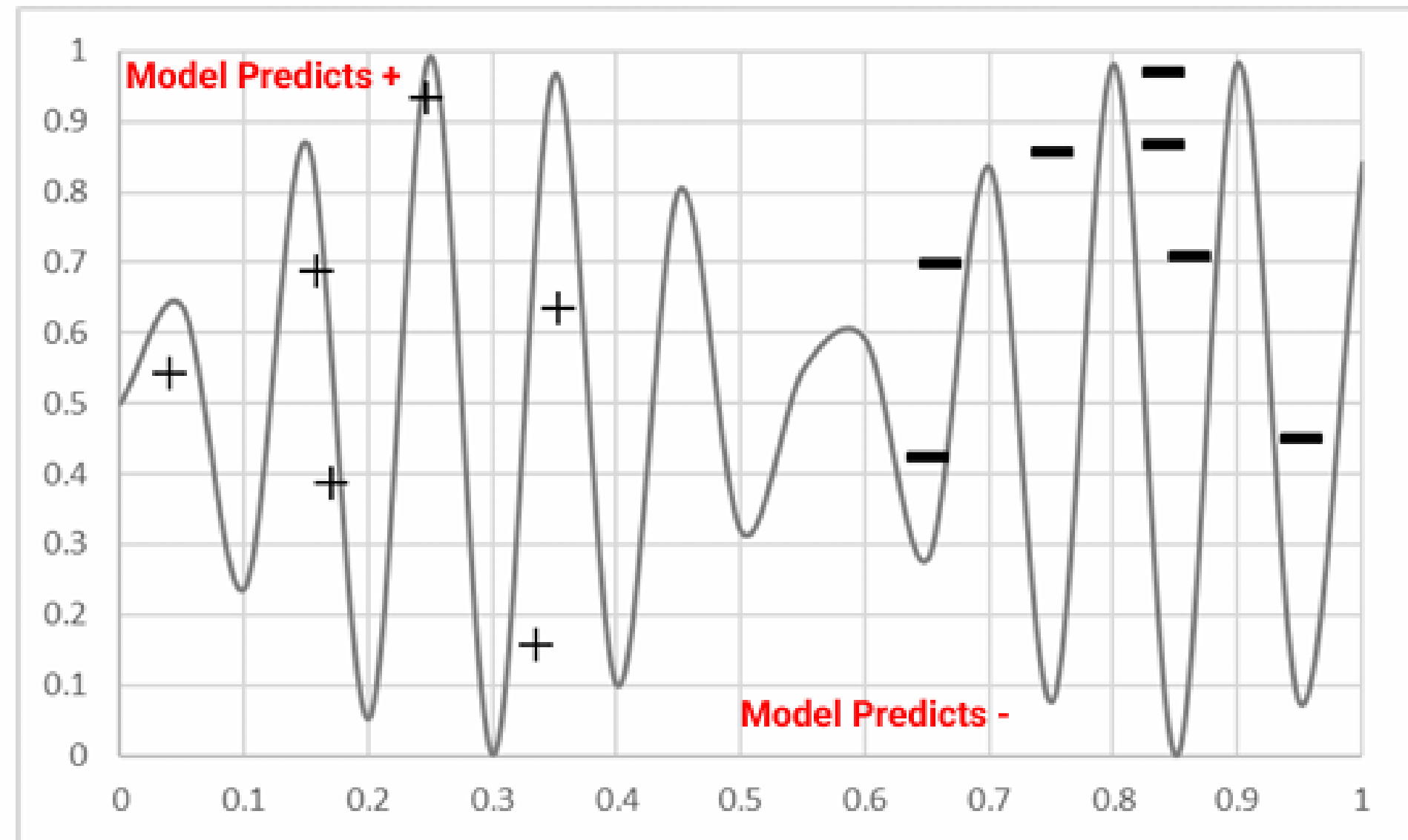
Bias and Variance: More Powerful Model

- Model yang kuat dapat merepresentasikan konsep-konsep yang kompleks
- Tidak ada Kesalahan!

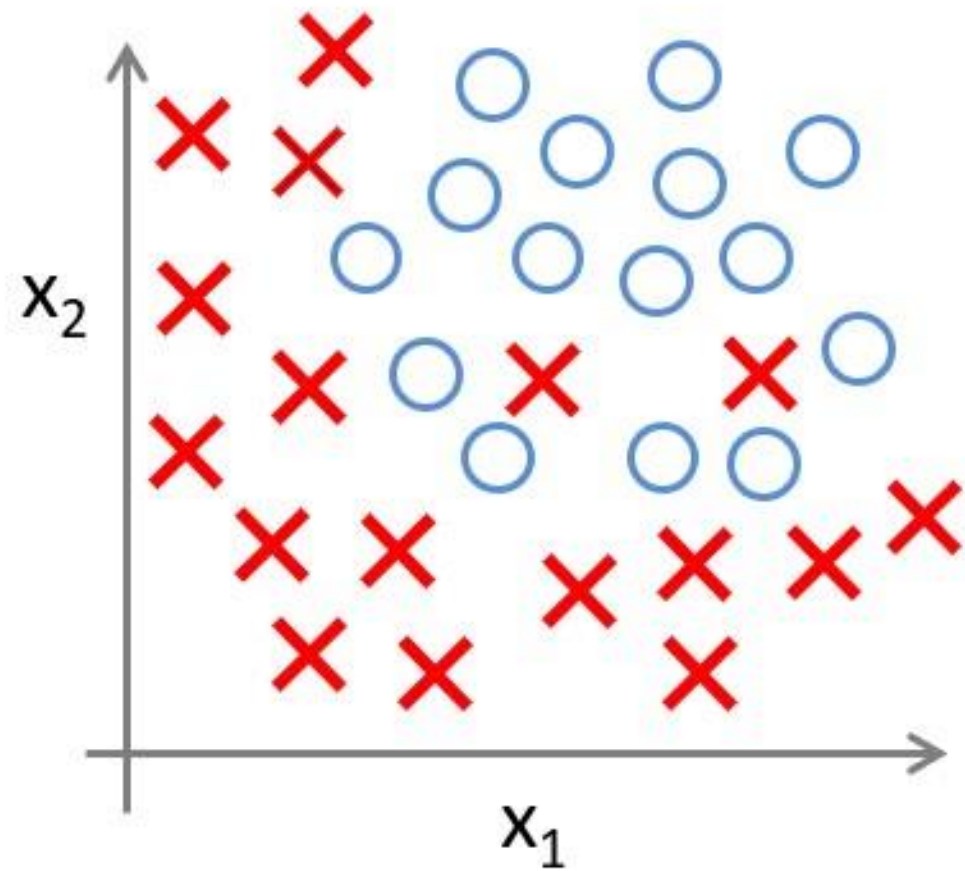


Bias and Variance: More Powerful Model

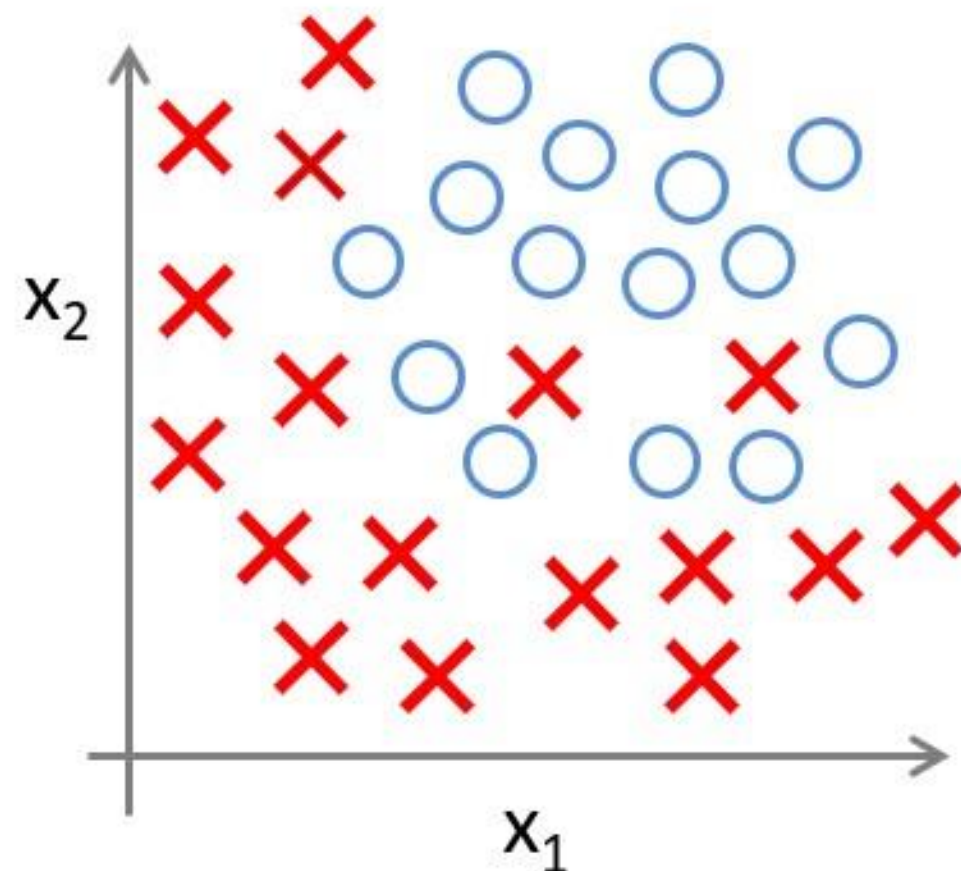
- Tapi jika ditambah lagi datanya
- Hasilnya tidak bagus



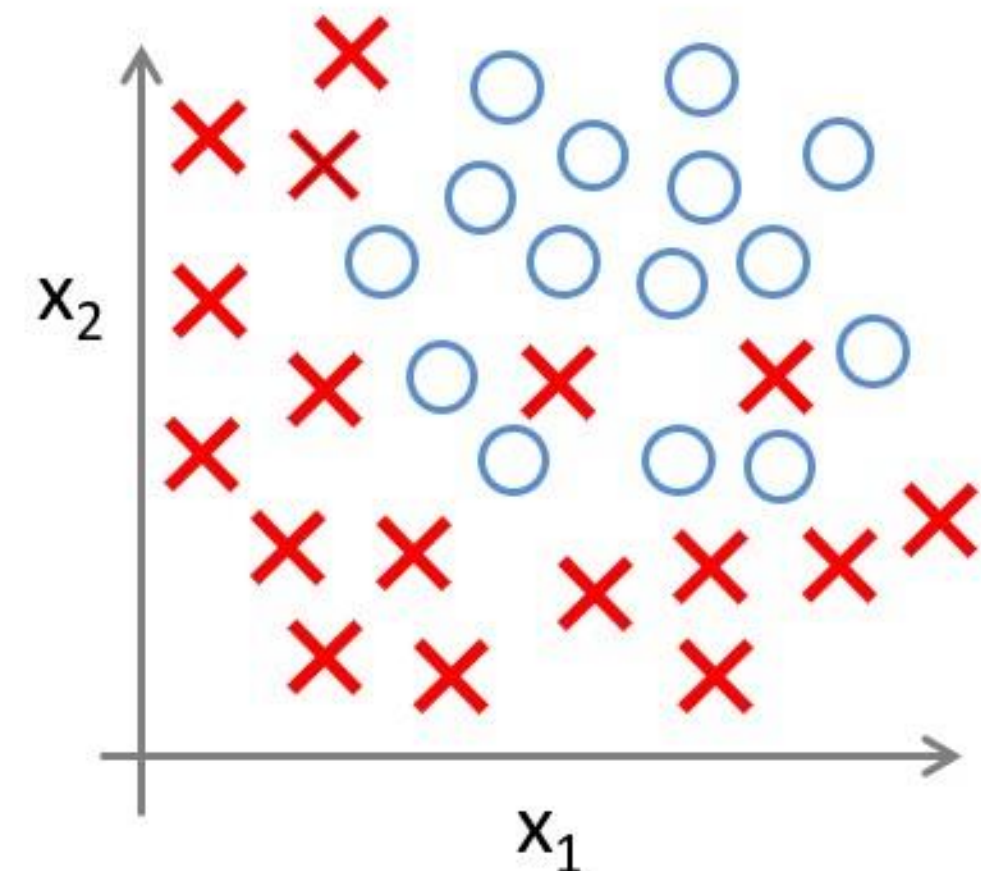
Bias and Variance



underfitting

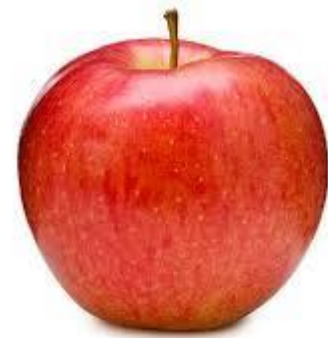


Just right



overfitting

Bias and Variance



Apel



Jeruk

Kondisi	Training Accuracy	Validation Accuracy	Diagnosis
Underfitting	60%	65%	Sama-sama rendah
Just Right	85%	82%	Gap kecil (normal)
Overfitting	99%	65%	Gap besar!

Overfitting vs Underfitting

Overfitting (High Variance, Low Bias)

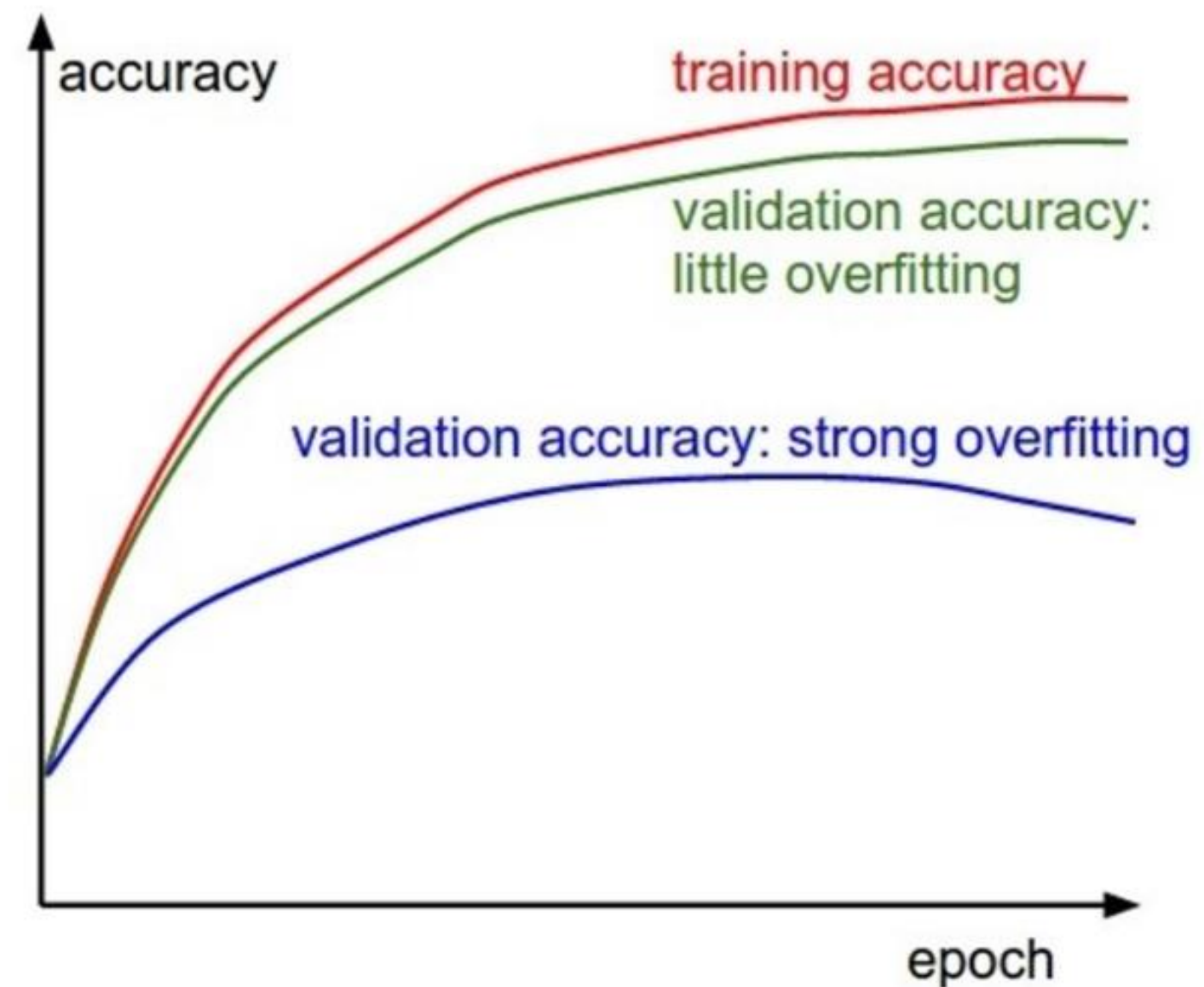
Model terlalu menyesuaikan diri dengan data training hingga mempelajari noise dan detail yang tidak relevan

Karakteristik:

- Training accuracy sangat tinggi jika dibandingkan dengan akurasi uji/validasi (Gap yang terlalu besar)
- Decision Boundary berbelit-belit mengikuti setiap data point

Penyebab:

- Terlalu banyak fitur dibandingkan jumlah data
- Model terlalu kompleks
- Training terlalu lama



Overfitting vs Underfitting

Underfitting (High Bias, Low Variance)

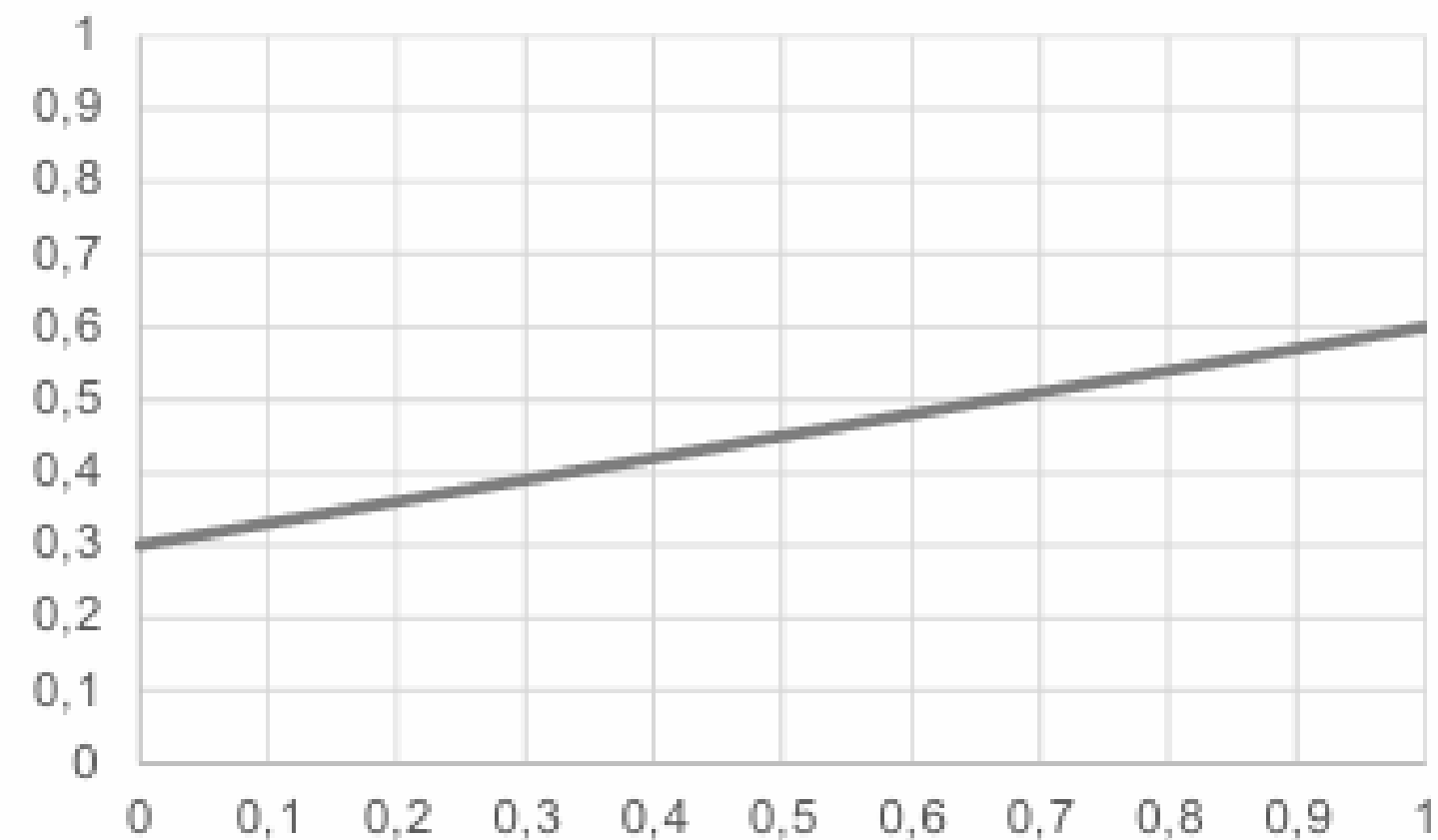
Model terlalu sederhana dan tidak dapat menangkap pola yang mendasari dalam data

Karakteristik:

- Training accuracy rendah, test akurasi juga rendah
- Model terlalu sederhana untuk data yang kompleks

Penyebab:

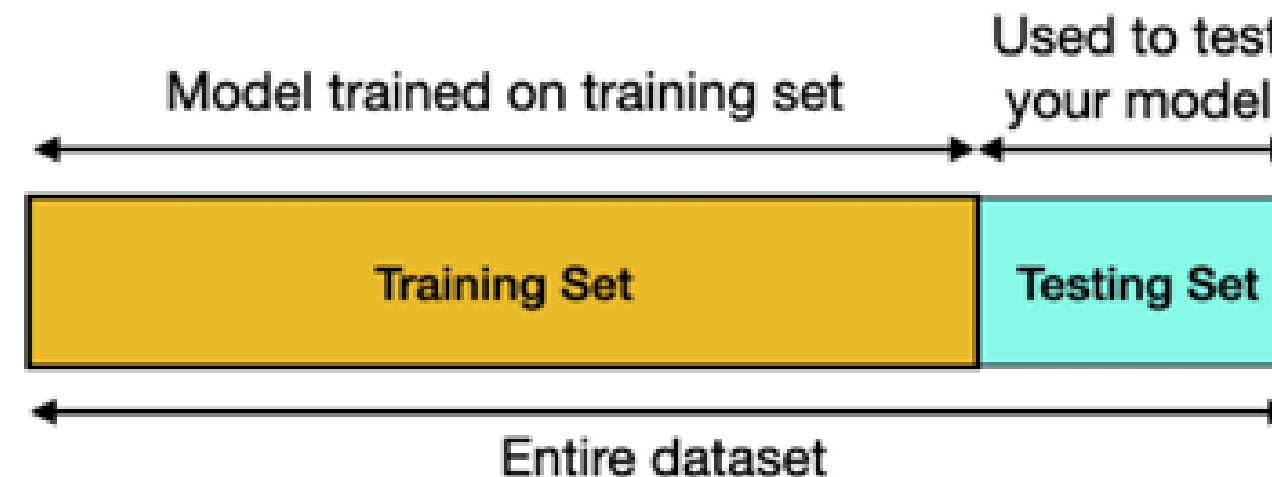
- Model terlalu sederhana untuk data yang kompleks
- Terlalu sedikit fitur
- Training yang tidak cukup



Avoiding Overfitting

Split the Dataset (Hold-Out)

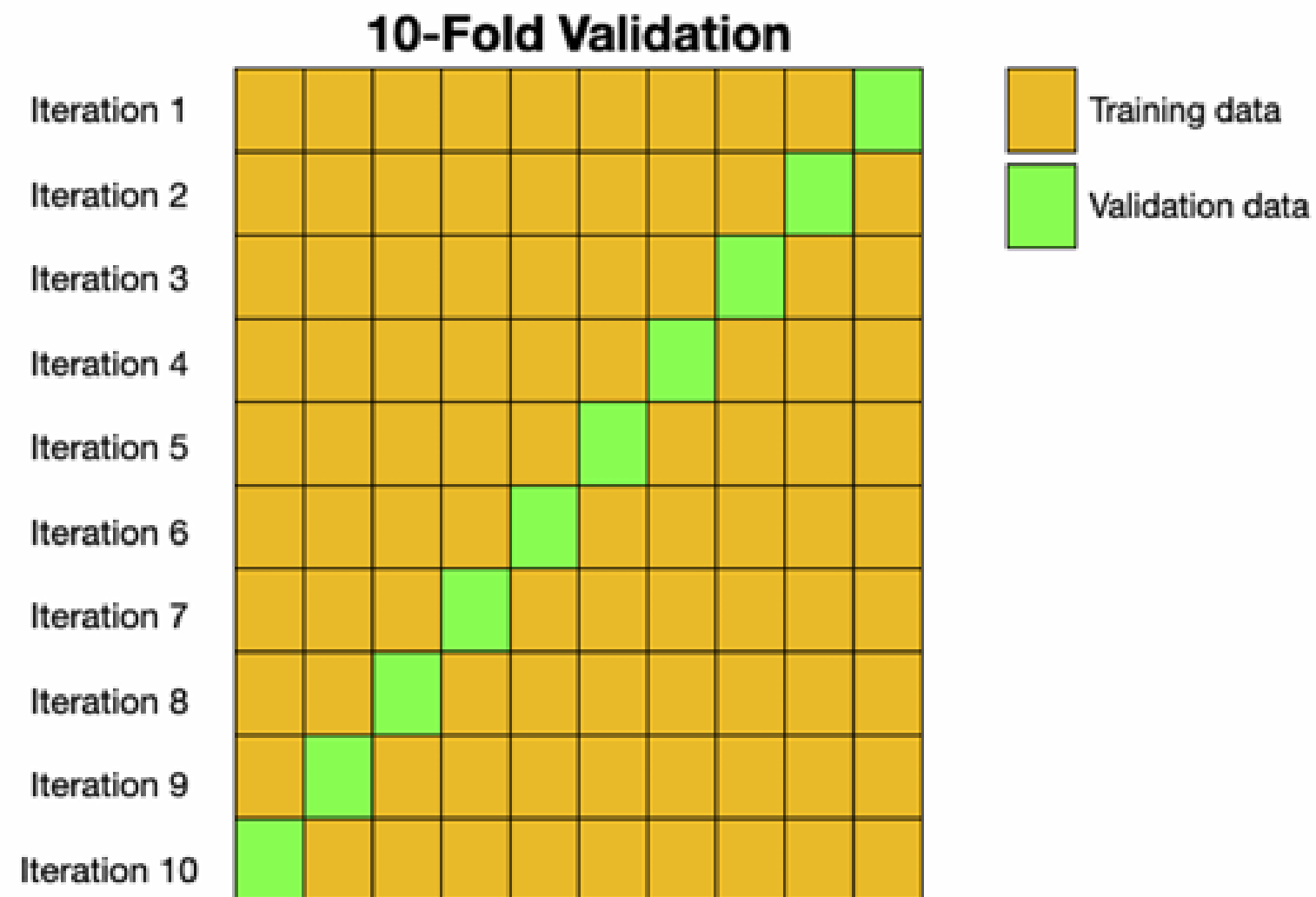
Konsep: Split dataset menjadi training dan testing set



Set **pengujian** digunakan untuk **mengevaluasi kinerja model** yang telah dilatih menggunakan set pelatihan. Meskipun ini adalah cara yang baik untuk mengevaluasi model, hal ini mungkin tidak memberikan indikasi yang sebenarnya tentang kinerja model. **Karna** Hasil bisa bias tergantung pada data yang terpilih untuk testing

Avoiding Overfitting

Cross Validation

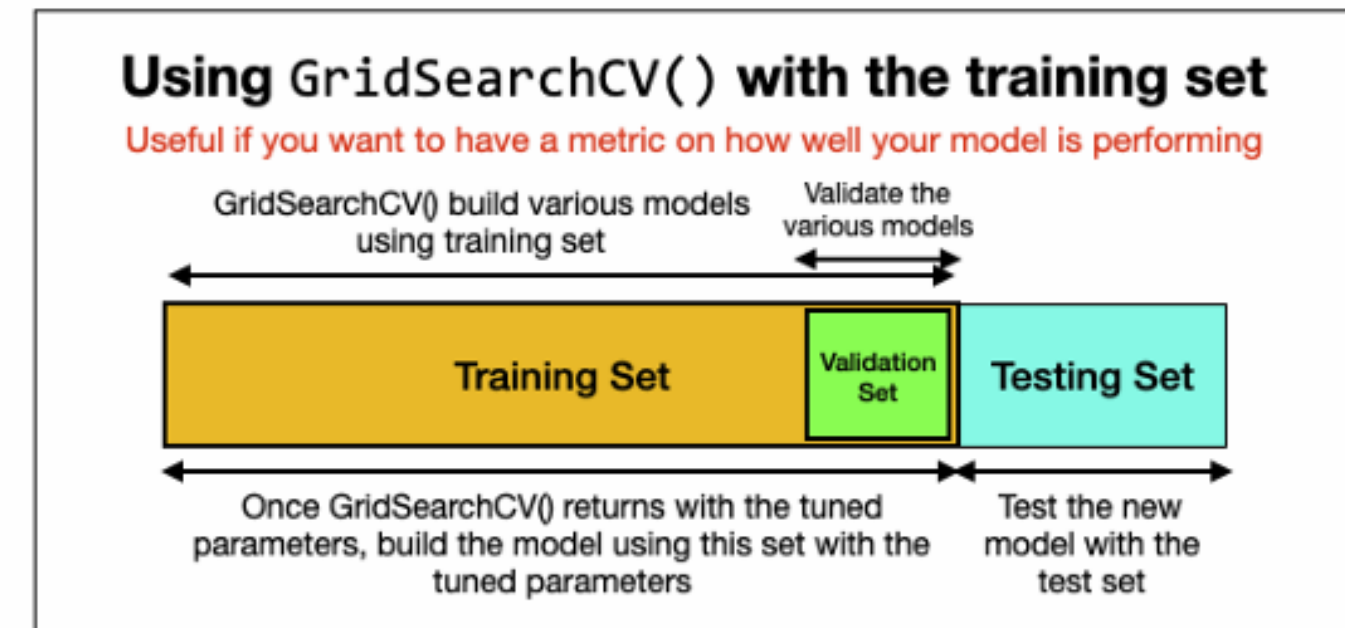


- Sebuah cara yang jauh lebih baik adalah membagi seluruh set data menjadi k-lipat (atau k-bagian, yaitu fold berarti membagi set data menjadi 10 bagian yang sama).
- Dalam setiap iterasi, metrik (seperti akurasi, presisi, dll) dan pada akhir semua iterasi, akan menghitung rata-rata dari metrik ini.
- Ini memberikan model Anda campuran data yang baik untuk pelatihan dan pengujian, dan memberikan tolok ukur yang lebih baik untuk kinerja model Anda.
- Proses membagi data menjadi k-fold dan menggunakan 1 fold untuk pengujian dan k-1 fold untuk pelatihan dikenal sebagai **k-fold cross validation**.

Avoiding Overfitting

GridSearchCV

- Ini melakukan pelatihan dan pengujian menggunakan validasi silang dari dataset Anda - sehingga akronim “CV” dalam GridSearchCV
- Pertama, buatlah grid parameter, yang merupakan kamus yang berisi semua berbagai hiperparameter yang ingin Anda coba saat membangun model Anda. Kemudian, panggil fungsi GridSearchCV() menggunakan algoritma pembelajaran mesin yang Anda gunakan.
- Setelah Anda selesai dengan pelatihan, Anda sekarang dapat mencetak hiperparameter yang telah disetel serta akurasi pelatihan. Dengan nilai-nilai hiperparameter yang dikembalikan oleh fungsi GridSearchCV(),



Avoiding Overfitting

Dropout

- **Dropout** adalah teknik yang membuang neuron dari jaringan saraf atau ‘mengabaikan’ mereka selama pelatihan, dengan kata lain, neuron yang berbeda dihilangkan dari jaringan untuk sementara waktu.
- Dengan menggunakan dropout, setelah setiap iterasi, set neuron yang berbeda diaktifkan, untuk mencegah beberapa neuron mendominasi proses.
- Dropout rate: Berkisar antara 0 dan 1.
- Dropout diaktifkan saat training **bukan saat testing**

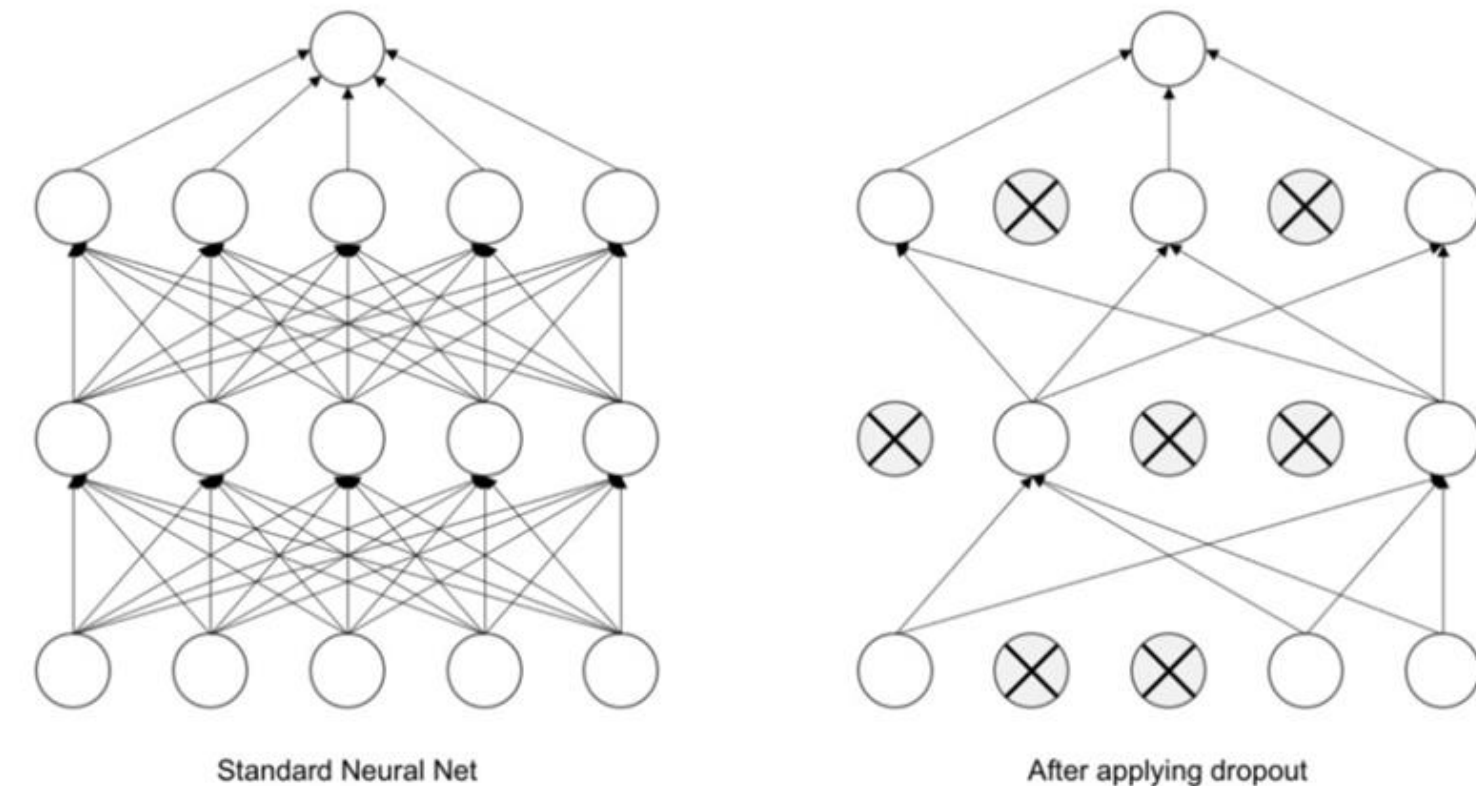


Image source: [Deep Learning for Computer Vision](#)

Paper: Dropout: A Simple Way to Prevent Neural Networks from Overfitting, Srivastata et. al, 2014

Avoiding Overfitting

Dropout

- **Dropout** adalah teknik yang membuang neuron dari jaringan saraf atau ‘mengabaikan’ mereka selama pelatihan, dengan kata lain, neuron yang berbeda dihilangkan dari jaringan untuk sementara waktu.
- Dengan menggunakan dropout, setelah setiap iterasi, set neuron yang berbeda diaktifkan, untuk mencegah beberapa neuron mendominasi proses.
- Dropout rate: Berkisar antara 0 dan 1.
- Dropout diaktifkan saat training **bukan saat testing**

```
model = keras.Sequential([
    # Convolutional layers
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),

    # Flatten untuk menuju fully connected layers
    layers.Flatten(),

    # Fully connected layers dengan dropout
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5), # Dropout rate 50%

    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3), # Dropout rate 30%

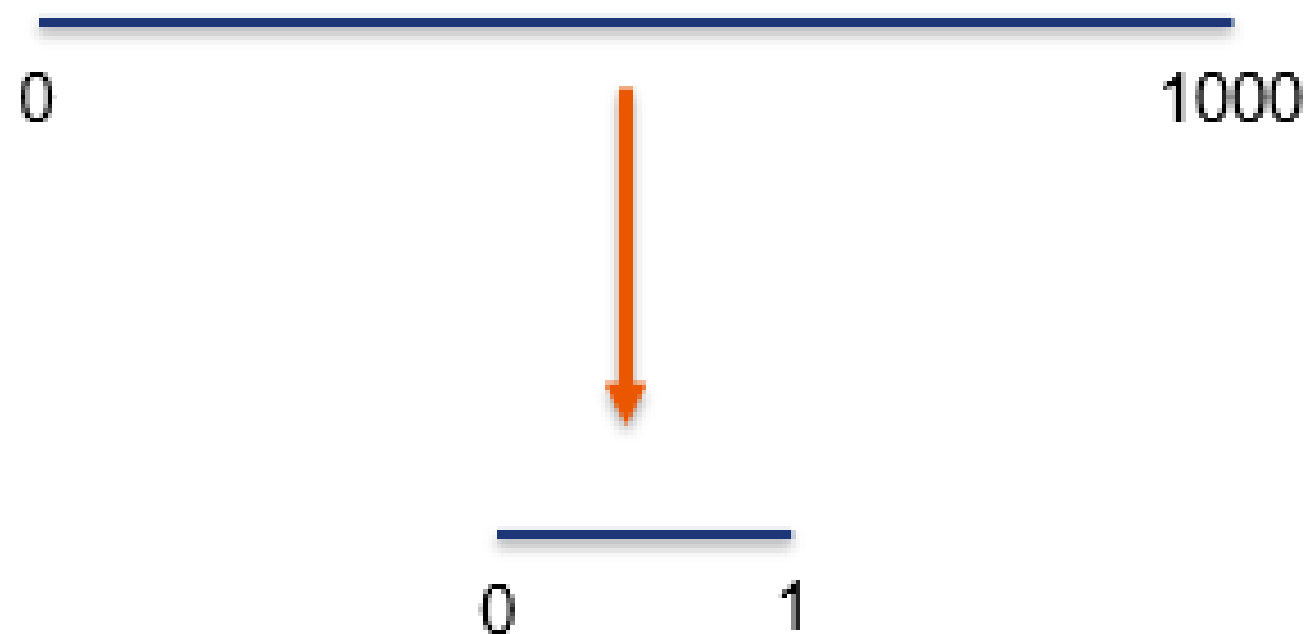
    # Output layer
    layers.Dense(10, activation='softmax')
])

return model
```

Avoiding Overfitting

Normalization

- **Normalisasi** memberikan bobot/kepentingan yang sama pada setiap variabel sehingga tidak ada variabel tunggal yang mengarahkan kinerja model ke satu arah hanya karena angka-angkanya lebih besar.



Min-Max Normalization

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

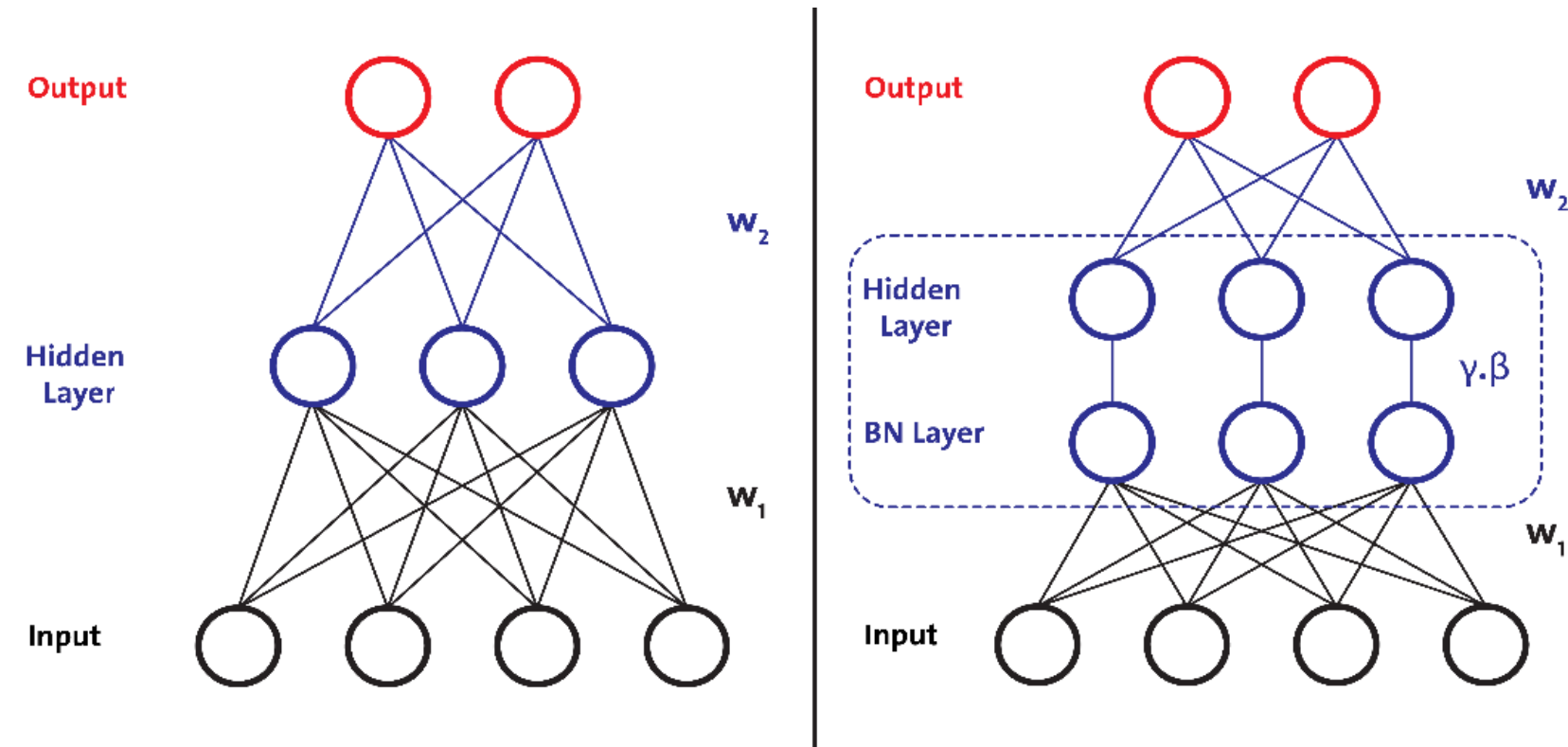
Z-Score Normalization

$$X' = \frac{X - \mu}{\sigma}$$

Avoiding Overfitting

Normalization

- Dalam jaringan saraf, keluaran lapisan pertama masuk ke lapisan kedua, keluaran lapisan kedua masuk ke lapisan ketiga, dan seterusnya.
- Ketika parameter lapisan berubah, distribusi masukan ke lapisan berikutnya pun berubah.
- Pergeseran dalam distribusi masukan ini dapat menjadi masalah bagi jaringan saraf, terutama jaringan saraf dalam yang dapat memiliki banyak lapisan.



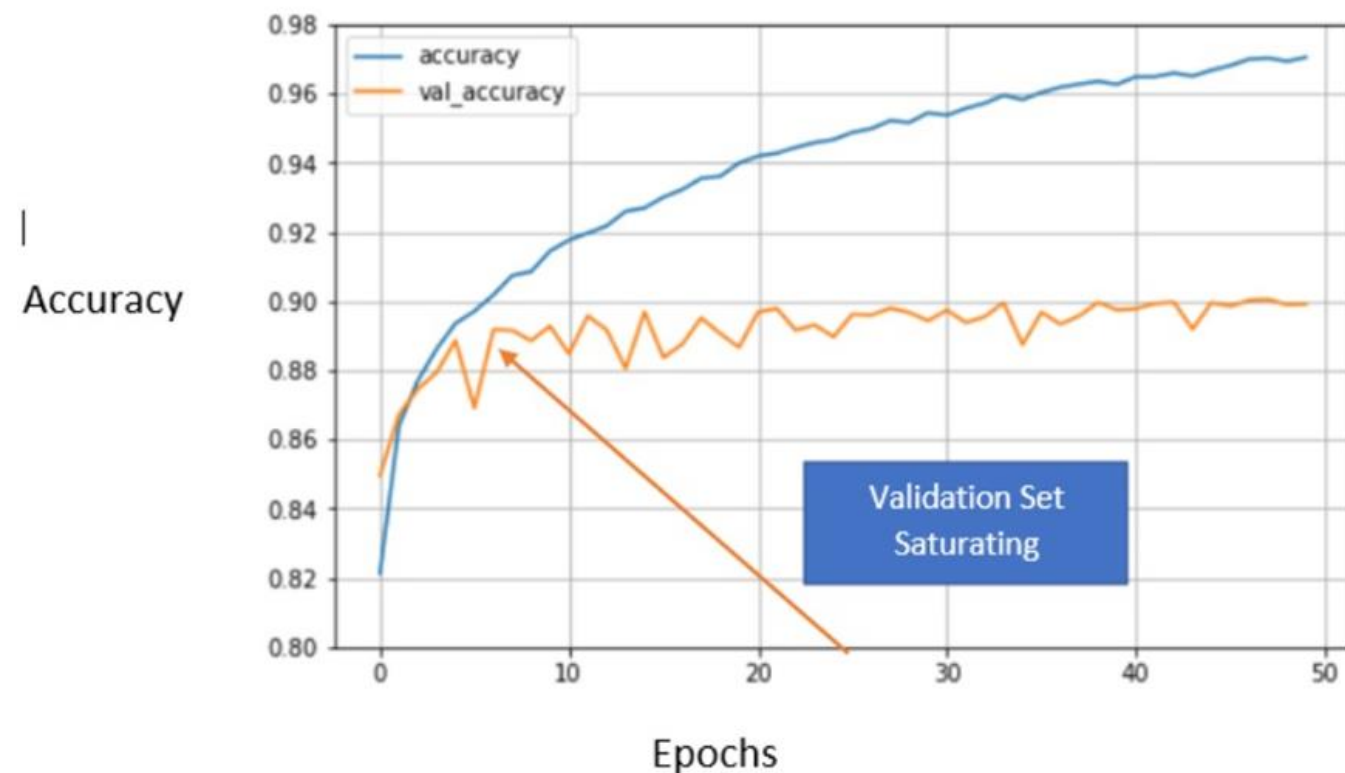
Source:
<https://www.oreilly.com/library/view/practical-convolutional-neural/9781788392303/928bb5f2-d518-4063-acd2-a87bdd786b6d.xhtml>
<https://machinelearning.wtf/terms/internal-covariate-shift/>

Avoiding Overfitting

Early Stopping

Early Stopping

- strategi penghentian dini berdasarkan kinerja set validasi.



Keakuratan set pelatihan terus meningkat, melalui semua Epoch. Namun, keakuratan set validasi mencapai titik jenuh antara 8 hingga 10 epoch. Di sinilah model dapat dihentikan pelatihannya.

```
early_stopping = EarlyStopping(  
    monitor='val_accuracy',  
    patience=3,           # Tunggu 3 epoch sebelum stop  
    restore_best_weights=True,  
    verbose=1  
)
```


Avoiding Underfitting

- **Tingkatkan kompleksitas model** - Model lebih fleksibel
- **Tambah fitur** - Feature engineering yang lebih baik
- **Training lebih lama** - Lebih banyak epoch untuk pembelajaran
- **Hapus noise dari data** - Data berkualitas lebih baik

Overcoming Data Imbalance

Mengatasi Data Imbalance

Data imbalance terjadi ketika distribusi kelas dalam dataset tidak seimbang, misalnya 95% kelas A dan hanya 5% kelas B. Masalah ini sangat umum dalam machine learning dan memerlukan penanganan khusus.

1. Resampling Techniques

- Oversampling dengan menambahkan jumlah sampel kelas minoritas. Teknik yang sering digunakan yaitu dengan SMOTE atau Random Oversampling (Menduplikasi sampel kelas minoritas secara acak).
- Undersampling dengan mengurangi jumlah sampel mayoritas secara acak. Teknik yang sering digunakan yaitu dengan Random Undersampling yaitu dengan menghapus sampel mayoritas secara acak.

2. Ensemble Methods

Overcoming Data Imbalance

Mengatasi Data Imbalance

Pada data gambar kita bisa menggunakan Teknik SMOTE atau **Augmentasi Data**.

- Teknik “**augmentasi data**” yang dikenal seperti Rotasi, flipping, scaling, cropping untuk kelas minoritas, bisa dilakukan untuk penanganan data tidak seimbang.
- Transfer Learning dan Fine-Tuning

Overcoming Data Imbalance

Penerapan Metode SMOTE Untuk Mengatasi *Imbalanced Data* Pada Klasifikasi Ujaran Kebencian

Ridwan¹, Eni Heni Hermaliani^{2*}, Muji Ernawati³

^{1,2,3}Universitas Nusa Mandiri
Jl. Kramat Raya No. 18 Jakarta Pusat, Indonesia

e-mail: ¹14210122@nusamandiri.ac.id, ^{2*}enie_h@nusamandiri.ac.id, ³14210225@nusamandiri.ac.id

(*) Corresponding Author

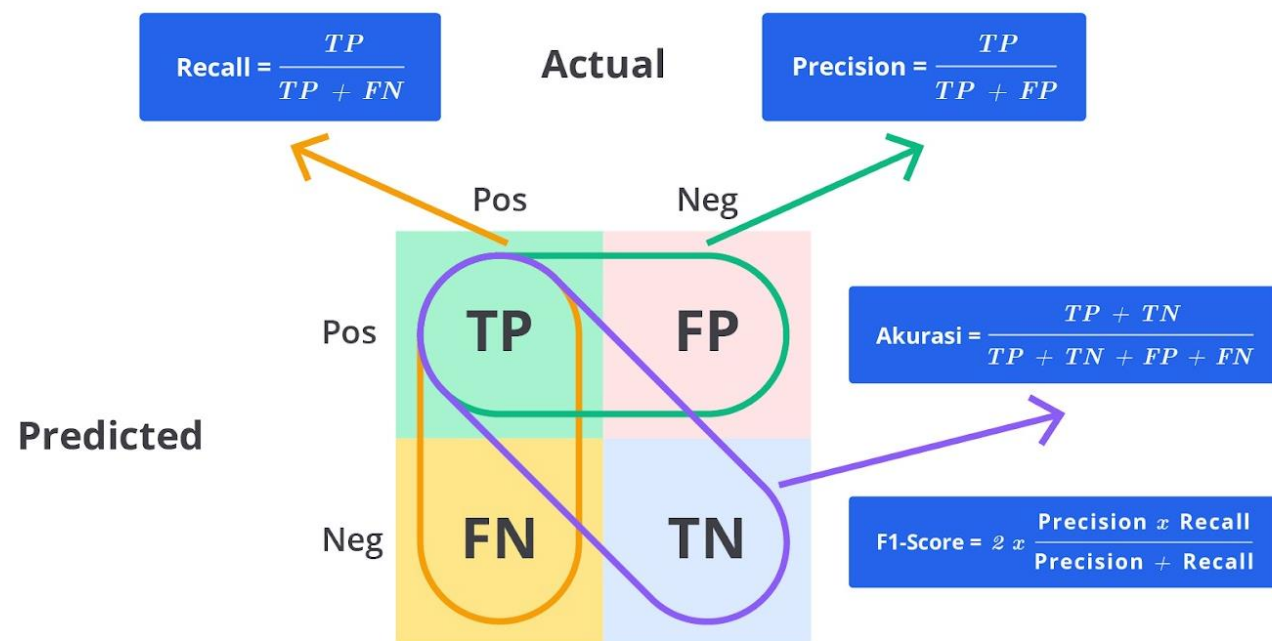
Artikel Info : Diterima : 01-12-2023 | Direvisi : 29-01-2024 | Disetujui : 31-01-2024

Abstrak - Ujaran kebencian adalah penyebaran kebencian terhadap individu atau kelompok atas dasar suku, agama, ras, dan karakteristik lain yang dapat menimbulkan diskriminasi, kekerasan, dan konflik sosial. Data yang tidak seimbang dapat menyebabkan hasil negatif pada hasil klasifikasi. Metode *Synthetic Minority Oversampling Technique* (SMOTE) digunakan untuk mengatasi data yang tidak seimbang. Ekstraksi fitur menggunakan *Bag of Words* dan TD-IDF, kemudian *data training* dilakukan *oversampling* menggunakan metode SMOTE, SVM-SMOTE, *Kmeans*-SMOTE, dan *Borderline*-SMOTE. Klasifikasi tersebut menggunakan algoritma *Random Forest*, *Support Vector Machine*, *Logistic Regression* dan *Naive Bayes* dengan menggunakan data *Twitter*. Hasil penelitian menunjukkan bahwa penerapan metode *Borderline*-SMOTE untuk menangani data *imbalanced* menghasilkan kinerja yang lebih baik daripada metode SMOTE lainnya berdasarkan nilai *accuracy*, *recall*, *precision* dan *F1-Score* dengan masing-masing nilai sebesar 84,09%, 85,25%, 84,55% dan 81,16%. Algoritma *Random Forest* menghasilkan nilai kinerja yang lebih tinggi dibandingkan algoritma lainnya.

Kata Kunci : Data *Imbalanced*, *Oversampling*, SMOTE, Ujaran Kebencian

Understanding Evaluation Metrics

Metrik Evaluasi Klasifikasi



F1-score menunjukkan performa sebenarnya dengan mempertimbangkan kedua aspek: seberapa baik model mendeteksi kelas minoritas (recall) dan seberapa akurat prediksi positifnya (precision).

Akurasi (Accuracy)

Definisi: Proporsi prediksi yang benar dari total prediksi

Formula: $(TP + TN) / (TP + TN + FP + FN)$

Presisi (Precision)

Definisi: Proporsi prediksi positif yang benar dari semua prediksi positif

Formula: $TP / (TP + FP)$

Recall (Sensitivity/True Positive Rate)

Definisi: Proporsi sampel positif yang berhasil dideteksi

Formula: $TP / (TP + FN)$

"Dari semua yang sebenarnya positif, berapa persen yang berhasil dideteksi?"

Recall sering digunakan sebagai tolak ukur untuk melihat apakah model itu bagus jika data yang digunakan adalah untuk diagnose penyakit

F1-Score

Definisi: Harmonic mean dari precision dan recall

Formula: $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

Practice

Image Classification: Deteksi Jenis Sampah di Sekitar Kampus

Dataset: <https://www.kaggle.com/datasets/farzadnekouei/trash-type-image-dataset>

Apakah model yang digunakan overfitting atau tidak? Jika iya lakukan Langkah yang sudah diajari bagaimana membuat model tidak overfitting.

Assignment

Buatlah ringkasan maksimal 2 halaman tentang inisialisasi bobot dan penyetelan hiperparameter dalam Deep Learning. Dikumpulkan ke elearning dalam bentuk pdf dengan format "NIM_Nama_Assignment11"

Reference

- <https://playground.tensorflow.org/>
- <https://github.com/gmortuza/Deep-Learning-Specialization/blob/master/2.%20Improving%20Deep%20Neural%20Networks:%20Hyperparameter%20tuning%2C%20Regularization%20and%20Optimization/week%201/Notes/Lecture.pdf>
- <https://ai-pool.com/a/s/dropout-in-deep-learning>
- <https://towardsdatascience.com/an-intuitive-explanation-to-dropout-749c7fb5395c>
- <https://www.analyticsvidhya.com/blog/2021/03/introduction-to-batch-normalization/>
- <https://towardsdatascience.com/what-is-batch-normalization-46058b4f583>
- https://keras.io/api/layers/regularization_layers/dropout/

THANK YOU!